Overview

Data structures and algorithms are core to AI engineering because they determine how quickly systems process data, how much memory they consume, and how well they scale under production traffic.

Real AI systems depend on these building blocks: - Queues for job schedulers and streaming consumers. - Stacks for parsing/evaluation workflows. - Hash maps for fast lookup in feature stores and aggregations. - Graph traversal for dependency analysis, recommendation relationships, and search spaces.

Core Data Structures

Arrays & Lists

Python lists provide dynamic arrays that support indexing, appending, slicing, and iteration. They are flexible but not always optimal for queue-like operations from the front.

Stacks & Queues

- A **stack** follows LIFO (last-in, first-out), useful for undo operations and depth-first style workflows.
- A **queue** follows FIFO (first-in, first-out), useful for task scheduling and breadth-first traversal.

Hash Maps (Dictionaries)

Dictionaries store key-value pairs with average O(1) lookup/update. They are essential for counting, caching, joins, and event aggregation in data/ML pipelines.

Trees & Graphs (High-Level)

Trees represent hierarchical structure; graphs represent general relationships. Many AI/search problems map naturally to graph traversal patterns.

Core Algorithms

Linear Search vs Binary Search

- **Linear search** scans elements one-by-one and works on unsorted data.
- **Binary search** repeatedly halves the search range, but requires sorted input.

Simple Sorting vs Built-in `sorted`

Simple algorithms like insertion sort are good for learning but can be slow on large inputs. Python's built-in `sorted` is highly optimized and should be preferred in production unless you have a specific constraint.

Traversal Algorithms (DFS/BFS)

- **DFS** explores deeply before backtracking.
- **BFS** explores layer by layer using a queue.

Both are widely used in dependency graphs, path exploration, and state-space search.

Big-O Complexity (Intuition)

Think of Big-O as growth behavior when input size increases: - **O(1):** constant-time operations (e.g., average dict lookup). - **O(log n):** grows slowly (binary search). - **O(n):** linear growth (single pass scan). - **O(n log n):** typical efficient sorting behavior. - **O(n^2):** quadratic growth; can become expensive quickly.

In production AI systems, complexity directly affects latency, cloud cost, and user experience.

Business Logic & Exceptions

Examples in real systems: - A naive O(n^2) matching loop inside a recommendation stage can become unusable as catalog size grows. - Recursive DFS can hit recursion limits on deep graphs if depth is uncontrolled. - Memory can blow up when using large eager lists instead of streaming/generator patterns.

Practical guidance for AI engineers: - Validate input sizes before expensive operations. - Prefer built-in optimized primitives when possible. - Choose structures based on access patterns (lookup-heavy -> dict, queue-heavy -> deque). - Add guardrails (timeouts, depth limits, memory checks) for traversal-heavy pipelines.

Interview Prep Checklist

- Implement stack and queue behavior using Python built-ins.
- Explain binary search and why it is O(log n).
- Discuss when to use a hash map instead of a list.
- Explain BFS vs DFS and when each is a better fit.
- Compare naive sorting complexity with optimized built-in sorting.
- Explain why complexity matters for latency and cost in AI systems.